OPTIMA: Density-Aware Task Scheduling and Low-Latency IPC on Asymmetric Multiprocessors with Linux sched_ext

Risheendra MN
Systems Research
rishindra.hackbox@gmail.com

***Abstract*—**Asymmetric chip-multiprocessors, combining high-throughput Performance (P) cores and power-efficient Efficiency (E) cores with disparate compute capacities and private cache topologies, present fundamental challenges for general-purpose operating system schedulers. While the Linux kernel incorporates capacity-aware scheduling (`sched-capacity`) and Energy-Aware Scheduling (EAS) to account for core compute differentials, standard fair-share classes (CFS, EEVDF) balance execution primarily by task virtual runtime lag and runnable load rather than execution density ($\rho = w/p$). Furthermore, existing schedulers lack a synchronous IPC bypass, inducing cross-CCX cache invalidation and high tail latencies when real-time and compute-bound tasks contend for heterogeneous execution resources.

In this paper, we present **OPTIMA**, an extensible operating system scheduler implemented in Linux `sched_ext` eBPF that provides principled algorithmic scheduling foundations for asymmetric multiprocessors. OPTIMA synthesizes three foundational mechanisms directly within the kernel hot-path: (1) a density-based adaptation of Smith's Rule (Weighted Shortest Processing Time, WSPT) that provably minimizes total weighted completion time $\sum w_j C_j$ on dedicated single-core queues and serves as an online approximation heuristic across asymmetric cores; (2) an Online Admissible Preemption Filter (OAPF) evaluating a dimensionally consistent one-step dispatch delay proxy $\Delta\text{DelayProxy}(c) = w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c)$ in $\mathcal{O}(M_P)$ kernel time to suppress preemption thrashing and guarantee local delay reduction; and (3) a synchronous IPC zero-lag fast-path that routes waking communicating tasks to local dispatch queues, substantially reducing cross-CCX migrations (from 48.7% to 0.2%) and preserving L1/L2 cache locality. Using quadratic Lyapunov drift with an exact algebraic cross-term expansion, we formally prove that OPTIMA achieves conditional multi-queue stability under aggregate admissible capacity, while virtual deadline bounding ($v_j \leq t + 3\tau_{\max}$) protects batch tasks against starvation.

We evaluate OPTIMA across 16 diverse workloads (pro-audio DSP, high-frequency financial order matching, game physics server, API gateway fan-out, Redis caching, microbenchmarks, IPC ping-pong, and parallel Linux kernel compilation) against CFS/EEVDF, `scx_rusty`, and `scx_rlfifo`. OPTIMA exhibits a bimodal performance profile: delivering an **84.1% reduction in IPC ping-pong latency** (2.55 μs vs 16.06 μs on CFS) with a **6.3× throughput increase** (392,691 ops/s), a **93.6% increase in network loopback bandwidth** (111.47 Gbps), **zero audio underruns**, **zero game desyncs**, and a **94.6% reduction in worst-case real-time latency spikes**. Conversely, on parallel software compilation (`make kernel/`), OPTIMA takes 6.03s, matching other `sched_ext` baselines (5.67s–5.83s) while trailing native in-kernel CFS (3.29s) due to eBPF struct-ops trampoline overhead during fork/exec process storms, and shows moderate throughput regressions (14–30%) under high-concurrency request multiplexing (Redis, API Gateway). We present an in-depth architectural analysis characterizing these workload trade-offs and identifying the microarchitectural boundaries between specialized eBPF dispatch and in-kernel heuristics.

***Index Terms*—Operating System Scheduling, eBPF, sched_ext, Asymmetric Multiprocessors, Online Preemption Filtering, WSPT, Lyapunov Stability, Real-Time Systems.**

I. Introduction

The relentless demands of modern computing workloads—ranging from sub-millisecond audio digital signal processing and algorithmic trading to massively parallel software compilation and microservices—have driven microarchitectural design toward single-ISA asymmetric multiprocessors (AMP). Modern multi-core processors integrate heterogeneous core types: high-frequency, wide-issue Performance cores (P-cores) designed for single-thread burst performance, and high-density, power-efficient Efficiency cores (E-cores) designed to maximize parallel compute density. These cores typically feature disparate clock frequencies, asymmetrical instruction throughput per cycle (IPC), and non-uniform cache topologies organized into discrete core complex (CCX) clusters.

Despite these architectural advances, general-purpose operating system CPU scheduling remains rooted in symmetric multiprocessor (SMP) fair-share abstractions. Although Linux incorporates capacity-aware scheduling (`sched-capacity`) and Energy-Aware Scheduling (EAS) on supported platforms to track CPU capacity differentials, the default fair-share class (CFS [1] and EEVDF [2], [3]) balances execution primarily by task virtual runtime lag and runnable load rather than execution density ($\rho = w/p$). Consequently, existing kernel schedulers suffer from three distinct operational limitations under heterogeneous workloads:

1) **Capacity and Density Mismatch:** While capacity-fitness checks migrate heavy tasks toward higher-capacity cores, they lack an execution-density metric to co-optimize priority weights against observed burst durations. High-priority interactive threads can experience capacity inversions or queueing delays behind batch jobs on Performance cores.
2) **Context-Switch Thrashing vs. Tail Latency:** When latency-critical tasks wake, CFS's virtual runtime preemption heuristics lack a dimensionally consistent preemption filter that accounts for context-switch and cache refill overheads, leading to either preemption storms or quantum wait spikes.

3) **Cross-Domain Cache Invalidation during Synchronous IPC:** In synchronous IPC and socket communication (e.g., pipe ping-pong, RPC fan-out), the default balancer searches for foreign idle cores across CCX boundaries, evicting hot L1/L2 caches and introducing costly inter-processor interrupt (IPI) latencies.

The recent introduction of `sched_ext` into upstream Linux 6.12 [4], [5] enables user-space and eBPF developers to replace the default kernel scheduling class with custom, dynamically loaded BPF struct-ops programs. While early sched-ext implementations, such as `scx_rusty` [5], `scx_lavd` [6], and the reference FIFO baseline `scx_rlfifo` [7], demonstrate the viability of eBPF scheduling, they rely primarily on heuristic balancing, user-space coordination daemons, or empirical policies that lack formal mathematical guarantees.

In this paper, we present **OPTIMA**, an extensible eBPF operating system scheduler that resolves these challenges through formal algorithmic synthesis. OPTIMA introduces mathematically grounded scheduling invariants directly into the kernel execution path:

- **Density-Based WSPT Sequencing (Smith's Rule):** We prove that ordering tasks by execution density $\rho_j = w_j/p_j$ is optimal for single-core dedicated queues ($1||\sum w_j C_j$), with ties permitted. While the general asymmetric problem with release dates ($R|r_j, \text{pmtn}|\sum w_j C_j$) is strongly NP-hard, OPTIMA leverages classical identical-machine results (Kawaguchi-Kyan reference bound) to formulate density-based virtual deadline scheduling as an online approximation heuristic, maximizing instantaneous backlog weight depletion via the Rearrangement Inequality.
- **Online Admissible Preemption Filtering (OAPF):** Rather than unconstrained heuristics, OPTIMA evaluates an online preemption filter over candidate Performance cores using the dimensionally consistent delay metric $\Delta\text{DelayProxy}(c) = w_{\text{curr}(c)} \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c)$. In $\mathcal{O}(M_P)$ bounded kernel time, OPTIMA provably preempts only when the one-step weighted delay is strictly reduced, suppressing preemption thrashing and enforcing bounded priority inversion ($\leq \tau_{\max}$).
- **Density-Guided Heterogeneous-Core Placement:** Core allocation between P- and E-cores is formulated via continuous fluid relaxation, preserving P-cores for high-density burst tasks and offloading long-running batch jobs ($p_j \geq 3\,\text{ms}$) to Efficiency queues.
- **Synchronous IPC Zero-Lag Local Dispatch:** By intercepting synchronous wakeup flags (`SCX_WAKE_SYNC`) and dispatching communicating partners directly into local hardware runqueues (`SCX_DSQ_LOCAL`), OPTIMA achieves 2.55 μs IPC turnaround and 392k ops/s throughput.
- **Conditional Lyapunov Stability & Starvation Protection:** We prove that adaptive quantum scaling combined with work-conserving work-stealing guarantees negative Lyapunov drift under a per-queue service condition, while virtual deadline bounding ($v_j \leq t + 3\tau_{\max}$) protects batch tasks against starvation.

II. Related Work & Comparative Landscape

A. Extensible and User-Space Operating System Schedulers

Extensible operating system scheduling has a rich history. Early systems investigated microkernel-assisted and user-level scheduling to grant applications fine-grained control over execution priorities. Google ghOSt [8] demonstrated that delegating scheduling decisions to user-space daemons via shared-memory queues enables specialized microsecond-scale policies for warehouse workloads. However, ghOSt pays a communication penalty: cross-privilege context switches and thread yields incur 1–2 μs overheads.

Linux `sched_ext` [4], [5] overcomes this boundary by compiling verified BPF programs directly into kernel space. Early community schedulers explore distinct trade-offs: `scx_rusty` uses a hybrid model with a multi-threaded Rust user daemon coordinating BPF dispatch queues; `scx_lavd` [6] targets gaming and desktop interactivity through empirical critical-path deadline tracking; and `scx_rlfifo` [7] serves as a reference Round-Robin / FIFO baseline built on `scx_rustland_core`. OPTIMA differs fundamentally from these systems by enforcing formal combinatorial optimization algorithms directly in kernel bytecode without user-space daemon overhead.

B. General-Purpose Linux Kernel Scheduling

The Linux kernel has evolved through multiple scheduling paradigms. The Completely Fair Scheduler (CFS) [1] implemented Generalized Processor Sharing (GPS) [9] via red-black trees ordered by virtual runtime (v_{runtime}). While CFS guarantees asymptotic fairness on symmetric systems, its SMP load balancer relies on coarse heuristic averages that fail to account for asymmetric core speedups. Linux 6.6 introduced EEVDF [2], replacing CFS's latency heuristics with Stoica and Abdel-Wahab's formal virtual deadline theory [3], [10]. EEVDF provides provable lag bounds on symmetric machines, but does not address asymmetric core partitioning or synchronous IPC cache locality.

For hard real-time workloads, `SCHED_DEADLINE` [11] implements Earliest Deadline First (EDF) coupled with the Constant Bandwidth Server (CBS) [12], [13]. While CBS guarantees temporal isolation, it requires static reservations (Q, P) and rejects unadmitted workloads. Kolivas's BFS [14] and MuQSS [15] highlighted that desktop latency spikes stem from heuristic load-balancing jitter, advocating for deterministic deadline ordering.

C. Asymmetric Multiprocessor (AMP) Scheduling

Scheduling on single-ISA heterogeneous processors was pioneered by Kumar et al. [16], [17]. Subsequent systems explored architectural bias scheduling [18] and contention-aware scheduling [19]–[21], demonstrating that memory-intensive tasks should be isolated to prevent cache degradation. Eyerman and Eeckhout [22], [23] analyzed CPI speedup stacks for asymmetric architectures.

On mobile platforms, ARM Energy-Aware Scheduling (EAS) [24] and DynamIQ evaluate energy differentials (ΔE) via an explicit kernel energy model. EAS prioritizes power reduction and reverts to standard CFS balancing under heavy load. Intel Thread Director [25], [26] provides hardware instruction classification via the Hardware Feedback Interface (HFI), but leaves policy enactment to the OS. In the domain of parallel runtimes, Chen et al. [27], [28] developed Selective Asymmetry-Aware Work-Stealing (SAWS) and Locality-Aware Work-Stealing (LAWS), showing that indiscriminate work-stealing across asymmetric cores degrades performance.

D. Theoretical Foundations of Scheduling

Smith's Rule [29] established that sequencing independent tasks in non-increasing order of density $\rho_i = w_i/p_i$ strictly minimizes total weighted completion time on an isolated single-core queue ($1||\sum w_i C_i$). However, in generalized multiprocessing, Lenstra et al. [30] and Garey and Johnson [31] proved that scheduling with dynamic release dates, preemption, and asymmetric parallel machines ($R|r_j, \text{pmtn}|\sum w_j C_j$) is strongly NP-hard (while Bruno et al. [32] proved NP-completeness for precedence-constrained tasks on identical processors). Classical approximation bounds, such as the Kawaguchi-Kyan bound ($\alpha \leq (1+\sqrt{2})/2 \approx 1.207$) [33], apply strictly to identical parallel machines ($P||\sum w_j C_j$) without release dates. Consequently, OPTIMA utilizes density-based list scheduling as a principled online approximation heuristic.

In online scheduling theory, Hall et al. [34], Megow and Schulz [35], and Phillips et al. [36] studied list scheduling guided by WSPT and linear programming relaxations. For preemption, rather than an intractable multi-stage branch-and-bound search tree, OPTIMA implements an Online Admissible Preemption Filter (OAPF) evaluated in $\mathcal{O}(M_P)$ kernel time. Finally, Neely's stochastic network optimization and Lyapunov drift analysis [37]–[39] provide the theoretical machinery to prove queue stability and bounded delay in stochastic queuing systems. Table I summarizes how OPTIMA advances beyond existing schedulers.

III. System Architecture & Implementation

OPTIMA is structured as a dual-plane operating system scheduler: a user-space telemetry and topology controller implemented in Rust, and a high-performance in-kernel scheduling engine implemented in eBPF via `sched_ext` struct-ops.

A. Heterogeneous Multi-Queue DSQ Hierarchy

Rather than maintaining a single global runqueue or uncoordinated per-CPU queues, OPTIMA partitions execution state across a hierarchical dispatch queue (DSQ) topology:

- `DSQ_PERF` (64-bit ID `0x100`): Dedicated exclusively to hard real-time and latency-critical tasks ($w_j \geq 500$ or pro-audio DSP). Executed with absolute priority by Performance cores.
- `DSQ_SHARED_P` (64-bit ID `0x101`): Shared domestic queue for general interactive and high-density tasks executing within the Performance core CCX domain.
- `DSQ_SHARED_E` (64-bit ID `0x102`): Shared domestic queue for balanced and throughput-oriented tasks executing within the Efficiency core CCX domain.
- `DSQ_EFF` (64-bit ID `0x103`): Long-running batch compute tasks ($p_j \geq 3$ ms) demoted from P-cores.
- `SCX_DSQ_LOCAL`: Per-CPU hardware dispatch queue into which tasks are directly committed for immediate zero-latency dispatch without global lock contention.

B. Dynamic Topology Discovery

Upon loading, the user-space Rust controller inspects the sysfs microarchitectural hierarchy (`/sys/devices/system/cpu/cpu*/`). It extracts core frequency ceilings, physical package topologies, and hardware thread sibling maps. Cores are classified into Performance partition $\mathcal{P}$ and Efficiency partition $\mathcal{E}$. The controller initializes eBPF read-only global masks (`p_core_mask`, `e_core_mask`, `smt_sibling_map`), allowing the kernel BPF JIT compiler to perform dead-code elimination and optimize dispatch branches into constant bitwise operations.

C. Synchronous IPC Zero-Lag Fast-Path

In synchronous client-server patterns (e.g., pipes, Redis GET/SET, microservice RPCs), the waking task and waker execute in lockstep. OPTIMA intercepts the `SCX_WAKE_SYNC` flag in `ops.select_cpu`. If the calling CPU (`this_cpu`) is allowed and has an empty local runqueue, OPTIMA immediately places the waking task into `SCX_DSQ_LOCAL` of `this_cpu`. If `this_cpu` is occupied, it attempts the SMT sibling (`this_sib`) or previous CPU (`prev_cpu`). This substantially reduces cross-CCX migrations (from 48.7% to 0.2%) and associated IPI interrupts, cutting ping-pong latency from 16.06 μs down to 2.55 μs.

D. Hierarchical Work-Conserving Work-Stealing

To prevent core starvation during parallel compute bursts (e.g., parallel software compilation), OPTIMA implements an asymmetric work-stealing protocol in `ops.dispatch`. Idle P-cores prioritize `DSQ_PERF` and `DSQ_SHARED_P`, but steal from `DSQ_SHARED_E` and `DSQ_EFF` when domestic queues are empty. Conversely, idle E-cores aggressively drain `DSQ_SHARED_P` without threshold barriers, preventing core underutilization and mitigating starvation under asymmetric compute bursts.

IV. Mathematical Theory & Formal Guarantees

A. System Model and Problem Formulation

We consider an asymmetric multiprocessor system $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ with M_P Performance cores of capacity c_P and M_E Efficiency cores of capacity c_E, where $\mu = c_P/c_E > 1$. The aggregate compute capacity rate is:

$$C_{\text{total}} \triangleq M_P c_P + M_E c_E \tag{1}$$

Online tasks $\mathcal{J} = \{J_1, J_2, \dots\}$ arrive dynamically, specified by 5-tuples $J_j = (r_j, p_j, w_j, d_j, \mathcal{T}_j)$ with nominal processing

TABLE I
COMPREHENSIVE COMPARISON MATRIX: OPTIMA VS. STATE-OF-THE-ART OPERATING SYSTEM SCHEDULERS

Scheduler		Asymmetry Awareness	Synchronous IPC Fast-Path	Preemption Provability	Tail-Latency Guarantees	Memory Footprint	Reconfig Overhead
OPTIMA	*(This Work)*	Hardware-native multi-tier; dynamic sysfs topology detection; speedup $\mu = c_P/c_E$.	Direct `SCX_DSQ_LOCAL` on `SCX_WAKE_SYNC` (2.55 μs); SMT locality.	Online Admissible Preemption Filter (OAPF): $\Delta\text{DelayProxy} < 0$; bounded inversion ($\leq \tau_{\max}$).	Density-based WSPT + virtual deadlines; Lyapunov drift queue stabilization.	BPF `task_storage` (64 B); per-CPU array maps (16 KB total).	$\mathcal{O}(1)$ atomic BPF `.rodata` live tuning; lightweight user topology controller.
CFS / EEVDF	*(Linux Default)*	Capacity-aware heuristics (`sched-capacity`); lacks execution density metrics.	Traverses full scheduling domain; cross-CCX IPI latency overhead.	Heuristic latency target (CFS); SMP lag bounds only (EEVDF).	Proportional lag bounded; susceptible to tail latency spikes under asymmetric contention and cross-domain load balancing.	Embedded in kernel `task_struct`; `sched_domain` tree.	Coarse sysctl tunables; requires kernel recompilation for logic changes.
Google ghOSt	[8]	User-space policy dependent; lacks dynamic hardware speed models.	Cross-privilege boundary context switches (1–2 μs penalty).	Policy dependent; preemption delayed by user-agent wakeup.	Application-specific tail tuning; fragile under heavy preemption.	High; multi-MB shared memory queues and user thread pools.	Medium; requires updating user daemon or restarting agents.
scx_rusty	[5]	Coarse domain clustering; does not compute dynamic core speedups.	BPF dispatch for local tasks; falls back to user daemon on imbalance.	Heuristic work-stealing based on queue depth ratios; no proof.	Moderate variance; tail spikes during user-space rebalancing.	Dual footprint: BPF maps plus Rust runtime memory heaps.	Low/Medium; CLI parameters modifiable via IPC channels.
scx_rlfifo	[7]	Unaware; global shared queue without core differentiation.	Generic dispatch; lacks synchronous waker-affinity routing.	Round-robin / FIFO; periodic slice preemption without priority differentiation.	Susceptible to head-of-line blocking and severe tail spikes under mixed workloads.	Minimal BPF dispatch queue state (8 KB).	Static FIFO policy; requires restarting scheduler to modify parameters.
scx_lavd	[6]	Topology-informed; tracks core frequencies for gaming interactivity.	Wakes latency-critical render threads on warm cores.	Heuristic critical-path tracking; lacks combinatorial bounds.	Low frame-time jitter; can penalize background batch throughput.	Low; in-kernel BPF maps for critical path tracking.	Low; live tuning of target frame rate deadlines.
ARM EAS	[24]	Energy-model integrated; Capacity-Aware big.LITTLE scheduling.	Standard energy differential (ΔE) evaluation; no IPC bypass.	None; preemption driven by standard CFS heuristic rules.	Energy prioritized; defers to CFS load balancer under load.	Embedded in kernel source; static ACPI/device-tree tables.	Rigid; modifying models requires device-tree reconfiguration.

requirement p_j (such that executing on core C_k with capacity c_k requires execution duration p_j/c_k), task density $\rho_j = w_j/p_j$, and priority weight $w_j \in [w_{\min}, w_{\max}]$ ($w_{\min} \geq 10$). The primary scheduling objective is to minimize total weighted completion time:

$$\min_{\mathcal{S}} \quad \Phi(\mathcal{S}) \triangleq \sum_{J_j \in \mathcal{J}} w_j C_j(\mathcal{S}) \tag{2}$$

subject to causality ($S_j \geq r_j$) and core exclusivity ($\sum_{J_j} \mathbf{1}_{\{J_j \text{ on } C_k \text{ at } t\}} \leq 1$).

Remark 1 (Latency-Sensitive Prioritization and Real-Time Scope)**.** *In an open, general-purpose operating system without hard admission control, arbitrary arrival bursts can exceed hardware processing capacity. Under unconstrained overload, guaranteeing hard real-time deadlines ($C_j \leq d_j$) for all tasks is mathematically impossible. OPTIMA provides latency-critical prioritization with zero-lag local dispatch (`SCX_DSQ_LOCAL`) and priority virtual deadlines ($v_j = t$) for latency-sensitive tasks ($w_j \geq w_{\text{RT}}$), minimizing queueing delay under non-overloaded regimes, while providing graceful degradation under overload via density-weighted completion time heuristics.*

B. Theorem 1: WSPT Sequencing, NP-Hardness, and Asymmetric Partitioning

Theorem 1 (Smith's Rule Optimality on Single-Core Dedicated Queues)**.** *Let $\mathcal{J}_k = \{J_1, \ldots, J_n\}$ be a set of n independent tasks available simultaneously at time t_0 ($r_j = t_0$) on an isolated processor core $C_k \in \mathcal{M}$. Any non-preemptive schedule ordering tasks in non-increasing density order:*

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(n)} \tag{3}$$

minimizes the total weighted completion time $\sum_{j=1}^{n} w_j C_j$. (Arbitrary tie-breaking among tasks of equal density preserves identical objective value.)

Proof. Assume for contradiction that an optimal schedule $\mathcal{S}^*$ does not follow non-increasing density order. There must exist adjacent tasks J_a, J_b where J_a immediately precedes J_b with $\rho_a < \rho_b$, meaning $w_a p_b < w_b p_a$.

Let $t \geq t_0$ be the start time of J_a. Under $\mathcal{S}^*$, completion times are $C_a = t + \frac{p_a}{c_k}$ and $C_b = t + \frac{p_a + p_b}{c_k}$. Interchanging J_a and J_b produces schedule $\mathcal{S}'$ without affecting any other tasks. For the swapped pair:

$$\begin{aligned} \Delta\Phi &= \Big[w_b C_b(\mathcal{S}') + w_a C_a(\mathcal{S}')\Big] - \Big[w_a C_a(\mathcal{S}^*) + w_b C_b(\mathcal{S}^*)\Big] \\ &= \frac{w_a p_b - w_b p_a}{c_k} < 0 \end{aligned} \tag{4}$$

Because $\Delta\Phi < 0$, $\mathcal{S}'$ achieves a strictly lower cost, contradicting optimality of $\mathcal{S}^*$. Finite adjacent transpositions transform any schedule into monotonic density order, establishing that Smith's ordering is optimal for single-core non-preemptive queues at time zero. □

Proposition 1 (Computational Complexity & Online Information Barrier)**.** *Minimizing total weighted completion time with dynamic release dates on parallel machines ($P|r_j|\sum w_jC_j$ or $R|r_j,pmtn|\sum w_jC_j$) is strongly NP-hard [30], [31]. Even on a single core, preemptive scheduling with arbitrary release dates ($1|r_j,pmtn|\sum w_jC_j$) is strongly NP-hard [30]. Furthermore, for online scheduling with unannounced arrivals, no online algorithm can achieve an optimal offline schedule regardless of computational power, as online decision-makers lack knowledge of future arrival sequences.*

Remark 2 (Context of the Kawaguchi-Kyan Bound)**.** *Kawaguchi and Kyan [33] proved that for identical parallel machines without release dates ($P||\sum w_jC_j$), WSPT list scheduling achieves an approximation factor of $\alpha \leq (1+\sqrt{2})/2 \approx 1.207$. This bound applies strictly to identical machines (P) at time zero. For asymmetric architectures (Q or R) and dynamic arrivals (r_j), OPTIMA uses density-based virtual deadline list scheduling as a principled online approximation heuristic.*

Theorem 2 (Continuous Fluid Relaxation for Density-Guided Placement)**.** *Under a continuous fluid relaxation where active jobs make continuous progress and priority weight is retired at a rate proportional to instantaneous compute bandwidth, the fluid weight retirement rate is $-\dot{W}_{\text{fluid}}(t) = \sum_{k=1}^{M} \rho_{\sigma(k)} c_k$. By the Hardy-Littlewood-Pólya Rearrangement Inequality, this fluid retirement rate is maximized when tasks with higher density are matched monotonically to faster cores:*

$$\min_{k\in\mathcal{P}} \rho_{\sigma(k)} \geq \max_{k'\in\mathcal{E}} \rho_{\sigma(k')} \tag{5}$$

Definition 1 (Task Density, Reference Normalization ρ_{ref}, and Virtual Deadline Increment)**.** *For each task J_j with priority weight $w_j \in [w_{\min}, w_{\max}]$ ($w_{\min} \geq 10$) and estimated burst runtime $p_{\text{est},j} \triangleq \max(\text{avg_runtime}_j, \tau_{\min})$, its execution density is defined as $\rho_j \triangleq \frac{w_j}{p_{\text{est},j}}$ with physical dimensions of $[\text{weight}/\text{time}]$ (or $[\mu\text{s}^{-1}]$). To preserve dimensional consistency when translating density into temporal virtual deadline advances, we introduce an explicit reference density $\rho_{\text{ref}} \triangleq \frac{w_{\text{ref}}}{p_{\text{ref}}} = \frac{100}{1000\,\mu\text{s}} = 0.1\,\mu\text{s}^{-1}$, representing a baseline interactive thread with default nice priority ($w_{\text{ref}} = 100$) and a 1 ms execution burst ($p_{\text{ref}} = 1000\,\mu\text{s}$). The virtual deadline increment assigned during enqueue is:*

$$\begin{aligned}\Delta v_j &\triangleq \text{clamp}\left(\tau_j \cdot \frac{\rho_{\text{ref}}}{\rho_j},\ \tau_{\min},\ \tau_{\max}\right) \\ &= \text{clamp}\left(\frac{\tau_j p_{\text{est},j}}{10\max(w_j, 10)},\ \tau_{\min},\ \tau_{\max}\right)\end{aligned} \tag{6}$$

Because the normalized density ratio ρ_{ref}/ρ_j is strictly dimensionless, Δv_j has exact physical dimensions of time $[\mu\text{s}]$, matching scheduling quantum τ_j.

Remark 3 (Decoupling Domain Placement from Intra-Queue Virtual-Deadline Ordering)**.** *We explicitly decouple coarse-grained domain placement from fine-grained intra-queue scheduling. First,* Domain Placement (Algorithm 3) *assigns tasks to target DSQs using heuristic thresholds: tasks with $w_j \geq w_{\text{RT}}$ route to* `DSQ_PERF`*, long-running batch jobs ($p_{\text{est},j} \geq 3\,\text{ms}$ or* `is_batch`*) route to* `DSQ_EFF`*, and general interactive tasks route to domestic shared queues (*`DSQ_SHARED_P` *or* `DSQ_SHARED_E`*) preserving CPU stickiness; domain placement does not evaluate $\rho = w/p$ directly. Second,* Intra-Queue Scheduling (Algorithm 3) *evaluates task density ρ_j and reference density ρ_{ref} exclusively within the virtual deadline calculus ($\Delta v_j \propto \rho_{\text{ref}}/\rho_j$) to establish the execution order ($\leq_v$) within shared dispatch queues, prioritizing higher-density tasks ahead of lower-density competitors.*

C. Theorem 3: Online Admissible Preemption Filtering (OAPF)

When a latency-critical task J_{wake} arrives and all P-cores are busy, OPTIMA evaluates an **Online Admissible Preemption Filter (OAPF)** greedy decision rule over candidate cores $c \in \mathcal{P}$ in $\mathcal{O}(M_P)$ kernel time, rather than an intractable multi-stage offline search tree.

Definition 2 (Weighted One-Step Dispatch Delay Proxy)**.** *Let $J_{\text{curr}}(c)$ be the task executing on core $c \in \mathcal{P}$ with weight $w_{\text{curr}}(c)$ and remaining quantum $R(c)$. Let waking task J_{wake} have weight w_{wake} and allocated time slice $\tau_{\text{slice}} = \tau_{\min}$. Let C_{switch} denote the context-switch and cache invalidation penalty in time units.*

If the scheduler waits, J_{wake} is delayed by $R(c)$, incurring weighted dispatch delay $w_{\text{wake}} \cdot R(c)$. If the scheduler preempts, $J_{\text{curr}}(c)$ is suspended during J_{wake}'s slice (τ_{slice}) plus context-switch overhead C_{switch}, incurring weighted delay penalty $w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}})$. The net differential in one-step weighted dispatch delay is:

$$\Delta\text{DelayProxy}(c) \triangleq w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c) \tag{7}$$

Both terms have unified physical dimensions of $[\text{weight} \times \text{time}]$ (priority-weighted delay units).

Theorem 3 (Local One-Step Dispatch Delay Reduction)**.** *Let $\mathcal{P}_{\text{admissible}} \subseteq \mathcal{P}$ be the set of candidate cores satisfying:*

1) *Priority superiority: $w_{\text{wake}} > w_{\text{curr}}(c)$;*
2) *Anti-thrashing remaining runtime: $R(c) > \delta_{\min}(c)$, where*

$$\delta_{\min}(c) \triangleq \frac{w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}})}{w_{\text{wake}}} \tag{8}$$

If $\mathcal{P}_{\text{admissible}} \neq \emptyset$, selecting core $c^* = \arg\min_{c\in\mathcal{P}_{\text{admissible}}} \Delta\text{DelayProxy}(c)$ and preempting $J_{\text{curr}}(c^*)$ guarantees that the one-step weighted dispatch delay is strictly reduced:

$$\Delta\text{DelayProxy}(c^*) < 0 \tag{9}$$

Conversely, if $\mathcal{P}_{\text{admissible}} = \emptyset$ or candidate cores satisfy $R(c) \leq \delta_{\min}(c)$, preemption is suppressed, bounding priority inversion to at most the running thread's quantum $\tau(c) \leq \tau_{\max}$.

Proof. Under non-preemptive wait, J_{wake} incurs waiting delay $R(c^*)$, while $J_{\text{curr}}(c^*)$ incurs 0 delay, yielding weighted delay $\text{Delay}_{\text{wait}} = w_{\text{wake}} R(c^*)$. Under preemption, J_{wake} incurs 0 dispatch delay, while $J_{\text{curr}}(c^*)$ incurs delay $(\tau_{\text{slice}} + C_{\text{switch}})$,

yielding weighted delay $\text{Delay}_{\text{preempt}} = w_{\text{curr}}(c^*)(\tau_{\text{slice}} + C_{\text{switch}})$. Computing $\Delta\text{DelayProxy}(c^*) = \text{Delay}_{\text{preempt}} - \text{Delay}_{\text{wait}}$ yields:

$$\Delta\text{DelayProxy}(c^*) = w_{\text{curr}}(c^*)(\tau_{\text{slice}}+C_{\text{switch}})-w_{\text{wake}}R(c^*) \tag{10}$$

Since $c^* \in \mathcal{P}_{\text{admissible}}$, $R(c^*) > \delta_{\min}(c^*)$, which implies $w_{\text{wake}}R(c^*) > w_{\text{curr}}(c^*)(\tau_{\text{slice}} + C_{\text{switch}})$, ensuring $\Delta\text{DelayProxy}(c^*) < 0$.

When preemption is suppressed (e.g., $w_{\text{curr}}(c) \geq w_{\text{wake}}$ or $R(c) \leq \delta_{\min}(c)$), the waking task waits for quantum expiration. Because each executing slice is clamped to $\tau(c) \leq \tau_{\max}$, the maximum waiting delay is bounded by $\tau_{\max}$, guaranteeing bounded priority inversion while protecting cache locality and pipeline execution. □

Remark 4 (Tunable Anti-Thrashing Dampening Penalty $C_{\text{switch}} = 50\,\mu$s)**.** *While raw CPU register context switches require only 1–2 µs and individual cache line reloads require tens to hundreds of nanoseconds, repeatedly displacing and reloading a memory-intensive working set introduces cumulative pipeline stalls across thousands of cycles. In OPTIMA,* C_{switch} *serves strictly as a* tunable anti-thrashing dampening penalty *nominally set to 50 µs (with an effective operational sensitivity range of 20–100 µs). Rather than modeling a single hardware cache miss,* C_{switch} *enforces an algorithmic hysteresis barrier, ensuring that preemption is triggered only when the waking task's priority advantage is sufficiently pronounced to justify evicting the incumbent thread and disrupting pipeline throughput.*

D. Theorem 4 & Proposition 2: Conditional Queue Stability and Starvation Protection

We model the multi-queue system state vector $\mathbf{Q}(t) = (q_1(t), q_2(t), q_3(t), q_4(t))^T$ evolving via $q_k(t+1) = \max(0, q_k(t) - s_k(t)) + a_k(t)$ with bounded second moments $\mathbb{E}[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)] \leq M_{\text{sec}} < \infty$. We define the quadratic Lyapunov function:

$$L(\mathbf{Q}(t)) \triangleq \frac{1}{2}\sum_{k=1}^{4} q_k(t)^2 \tag{11}$$

Definition 3 (Aggregate Admissible Workload Region)**.** *The arrival workload is aggregate-admissible if aggregate workload demand is within total multiprocessor compute capacity:*

$$\sum_{k=1}^{4} \lambda_k \bar{p}_k \leq C_{\text{total}} - \epsilon_0 = (M_P c_P + M_E c_E) - \epsilon_0, \quad \epsilon_0 > 0 \tag{12}$$

where $\lambda_k = \mathbb{E}[a_k(t)]$ *and* $\bar{p}_k$ *is mean task execution work.*

Theorem 4 (Conditional Multi-Queue Stability via Foster-Lyapunov Drift)**.** *Let the multi-queue backlog* $\mathbf{Q}(t)$ *evolve under an aggregate-admissible workload (Definition 3). If the dispatch and hierarchical work-stealing policy guarantees a positive service slack across all active non-empty queues:*

$$\mathbb{E}\big[s_k(t) - a_k(t) \;\big|\; \mathbf{Q}(t)\big] \geq \delta > 0, \quad \forall k : q_k(t) > 0 \tag{13}$$

then the quadratic Lyapunov function satisfies the negative drift condition:

$$\Delta L(\mathbf{Q}(t)) \leq B - \delta \sum_{k=1}^{4} q_k(t) \tag{14}$$

for finite positive constant $B = \frac{1}{2}\sum_{k=1}^{4} \mathbb{E}[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)] < \infty$*. Consequently, the multi-queue system is Foster-Lyapunov stable (positive recurrent), with bounded time-average queue depth:*

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4} \mathbb{E}[q_k(t)] \leq \frac{B}{\delta} < \infty \tag{15}$$

Proof. Expanding $q_k(t+1)^2$:

$$\begin{aligned} q_k(t+1)^2 &= (\max(0, q_k(t) - s_k(t)))^2 + a_k(t)^2 \\ &\quad + 2a_k(t)\max(0, q_k(t) - s_k(t)) \end{aligned} \tag{16}$$

Using $(\max(0,x))^2 \leq x^2$, we bound the square by $(q_k - s_k)^2 = q_k^2 - 2q_k s_k + s_k^2$. For the cross-term, since $a_k \geq 0$ and $\max(0, q_k - s_k) \leq q_k$, we have $2a_k \max(0, q_k - s_k) \leq 2q_k a_k$. Combining both bounds:

$$q_k(t+1)^2 \leq q_k(t)^2 + a_k(t)^2 + s_k(t)^2 - 2q_k(t)\big(s_k(t) - a_k(t)\big) \tag{17}$$

Subtracting $q_k(t)^2$, dividing by 2, and taking conditional expectations:

$$\Delta L(\mathbf{Q}(t)) \leq B - \sum_{k=1}^{4} q_k(t)\,\mathbb{E}\big[s_k(t) - a_k(t) \;\big|\; \mathbf{Q}(t)\big] \tag{18}$$

Under the stated per-queue service condition (13), $\mathbb{E}[s_k(t) - a_k(t) \mid \mathbf{Q}(t)] \geq \delta > 0$ for non-empty queues. Substituting this into the drift bound yields $\Delta L(\mathbf{Q}(t)) \leq B - \delta \sum_k q_k(t)$. By the Foster-Lyapunov stability criterion, this establishes positive recurrence and strong aggregate queue stability. □

Remark 5 (Architectural Realization of the Service Slack Condition)**.** *In multi-queue systems with strict priorities, global capacity admissibility* ($\sum \lambda_k \bar{p}_k \leq C_{\text{total}} - \epsilon_0$) *is a necessary condition for aggregate stability, but does not independently ensure per-queue service slack (13) if high-priority queues can monopolize all cores. OPTIMA establishes this per-queue condition through structural partitioning: (1) dedicated Efficiency cores provide guaranteed domestic compute capacity to batch queues (`DSQ_EFF`), preventing high-priority P-core traffic from completely starving throughput work; and (2) dynamic quantum scaling* $\tau(q_k) = \max(\frac{Q_{\text{base}}}{1+q_k}, \tau_{\min})$ *dynamically compresses time slices during congestion, accelerating task dispatch and preemption frequency.*

Proposition 2 (Virtual Deadline Bounding and Starvation Protection)**.** *Under OPTIMA's virtual deadline calculus (Algorithm 3), for every task* J_j *enqueued into a batch or interactive queue (`DSQ_SHARED_P`, `DSQ_SHARED_E`, `DSQ_EFF`) at time* t*, its assigned virtual deadline satisfies:*

$$v_j(t) = \text{clamp}(v_{\text{time}}, t, t + 2\tau_j) + \Delta v_j \leq t + 3\tau_{\max} \tag{19}$$

Algorithm 1 OPTIMA-SELECT-CPU (IPC Fast-Path & Domain Selection)

Require: Task p, previous CPU $prev_cpu$, wakeup flags $wake_flags$
Ensure: Target CPU $target_cpu$

$tctx \leftarrow$ lookup_task_ctx(p)
if p is pinned **or** p is kernel thread **then**
  Dispatch to $prev_cpu$ via `SCX_DSQ_LOCAL`; **return** $prev_cpu$
end if
$this_cpu \leftarrow$ smp_processor_id(), $idle_mask \leftarrow$ get_idle_smtmask()
if $wake_flags$ & SCX_WAKE_SYNC **then**
  if $this_cpu \in p$.cpus_allowed $\wedge$ dsq_nr_queued($this_cpu$) $== 0$ **then**
    goto direct_dispatch with $target_cpu \leftarrow this_cpu$
  end if
  $this_sib \leftarrow$ get_smt_sibling($this_cpu$)
  if $this_sib \in p$.cpus_allowed $\wedge$ dsq_nr_queued($this_sib$) $== 0$ **then**
    goto direct_dispatch with $target_cpu \leftarrow this_sib$
  end if
end if
if is_perf($prev_cpu$) $\wedge$ $prev_cpu \in idle_mask \wedge$ test_and_clear_idle($prev_cpu$) **then**
  goto direct_dispatch with $target_cpu \leftarrow prev_cpu$
end if
if $tctx.core_type \neq$ CORE_TYPE_EFF **then**
  $c \leftarrow$ pick_idle_domestic_core(p, is_perf = TRUE, $idle_mask$)
  if $c \geq 0$ **then**
    goto direct_dispatch with $target_cpu \leftarrow c$
  end if
end if
if $tctx.weight \geq w_{\mathrm{RT}}$ **then**
  $oapf_cpu \leftarrow$ OPTIMA-OAPF($p, tctx, \mathcal{P}$)
  if $oapf_cpu \geq 0$ **then**
    $tctx.dispatch_local \leftarrow$ TRUE; **return** $oapf_cpu$
  end if
end if
$tctx.dispatch_local \leftarrow$ FALSE; **return** $prev_cpu$
direct_dispatch:
Compute slice τ based on $tctx.is_batch$ and $tctx.avg_runtime$
scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, τ, 0)
return $target_cpu$

Algorithm 2 OPTIMA-OAPF (Online Admissible Preemption Filter)

Require: Waking task p, context $tctx$, candidate core set $\mathcal{P}$
Ensure: Target CPU c^* (or -1 if wait is optimal)

$c^* \leftarrow -1$, $\Delta\mathrm{Proxy}^* \leftarrow 0$, $now \leftarrow$ scx_bpf_now()
$w_{\mathrm{wake}} \leftarrow \max(tctx.weight, 100)$, $\tau_{\mathrm{slice}} \leftarrow \tau_{\min}$, $C_{\mathrm{switch}} \leftarrow 50\,\mu$s
for each core $c \in \mathcal{P}$ **do**
  if $c \notin p$.cpus_allowed **then**
    continue
  end if
  $st \leftarrow$ lookup_cpu_run_state(c)
  if $st ==$ NULL $\vee$ $st.pid == 0$ **then**
    return c {Rule 1: Idle core}
  end if
  if $st.weight \geq w_{\mathrm{wake}}$ **then**
    continue {Rule 2: Inversion safety}
  end if
  $el \leftarrow now - st.start_time$, $R(c) \leftarrow (st.slice > el)\,?\,(st.slice - el) : 0$
  $\delta_{\min} \leftarrow \frac{st.weight \cdot (\tau_{\mathrm{slice}} + C_{\mathrm{switch}})}{w_{\mathrm{wake}}}$
  if $R(c) \leq \delta_{\min}$ **then**
    continue {Rule 3: Anti-thrashing}
  end if
  $\Delta\mathrm{DelayProxy} \leftarrow st.weight \cdot (\tau_{\mathrm{slice}} + C_{\mathrm{switch}}) - w_{\mathrm{wake}} \cdot R(c)$
  if $\Delta\mathrm{DelayProxy} < \Delta\mathrm{Proxy}^*$ **then**
    $\Delta\mathrm{Proxy}^* \leftarrow \Delta\mathrm{DelayProxy}$; $c^* \leftarrow c$
  end if
end for
if $c^* \geq 0 \wedge \Delta\mathrm{Proxy}^* < 0$ **then**
  scx_bpf_kick_cpu(c^*, SCX_KICK_PREEMPT);
  return c^*
end if
return -1

since $\tau_j \leq \tau_{\max}$ and $\Delta v_j \leq \tau_{\max}$. Any subsequent batch task $J_{j'}$ arriving at timestamp $t' > t + 3\tau_{\max}$ receives $v_{j'}(t') \geq t' > v_j(t)$ and cannot be scheduled ahead of J_j. While a batch task arriving earlier ($t' \leq t + 3\tau_{\max}$) with higher density may receive an earlier deadline, $v_j(t) \leq t + 3\tau_{\max}$ strictly bounds relative queue advancement lag. Under the conditional service rate condition ($\mathbb{E}[s_k - a_k \mid \mathbf{Q}] \geq \delta > 0$) of Theorem 4, every admitted batch task is guaranteed dispatch within finite time. Furthermore, latency-critical tasks ($w \geq w_{\mathrm{RT}}$) receive immediate priority ($v_j = t$) and always supersede batch tasks; when preemption is suppressed by OAPF, the immediate waiting delay imposed by the running task is bounded by $\tau_{\max}$, ensuring bounded priority inversion.

E. Algorithmic Specifications

Algorithms 1, 2, 3, and 4 detail the exact execution logic compiled into kernel eBPF bytecode.

V. Empirical Evaluation

A. Experimental Methodology

We conduct all experiments on a modern heterogeneous asymmetric multi-core testbed comprising 12 hardware execution threads across 4 wide-issue Performance cores and 8 compact Efficiency cores sharing a unified multi-megabyte Level-3 cache hierarchy and dual-channel memory, running Linux 6.12 with `sched_ext`. Throughout our evaluation, the baseline labeled "CFS / EEVDF" denotes the Linux 6.12 mainline default fair scheduler class (which implements Earliest Eligible Virtual Deadline First proportional-lag allocation with standard CFS load-balancing heuristics). To eliminate external OS jitter and background interference, the evaluation harness executes within an isolated, hardware-partitioned execution domain:

- **Isolated Execution Partition:** Benchmark workloads execute across 12 dedicated hardware execution threads partitioned between Performance and Efficiency domains.
- **Hardware Cache Isolation:** Hardware Cache Allocation Technology (CAT) bitmasks partition L3 cache ways, strictly preventing cross-domain cache evictions between host monitoring infrastructure and benchmark threads.
- **Dedicated IRQ Routing:** External device interrupts are affinity-pinned away from benchmark cores.
- **Statistical Rigor:** Every workload is executed through 3 complete, independent repetitions. We report mean values, sample standard deviations (σ), and relative percentage deltas.

B. Benchmark Workload Taxonomy

We evaluate 16 diverse production and synthetic workloads covering real-time, throughput, memory, and communication profiles:

TABLE II
COMPREHENSIVE 3-REPETITION BENCHMARK RESULTS ACROSS 16 WORKLOADS (34 EVALUATED METRICS)

Benchmark Workload	Metric Evaluated	Unit	CFS / EEVDF	scx_optima	scx_rusty	scx_rlfifo	Advantage (vs CFS)
Pro-Audio Turnaround Latency	Buffer P50 (↓)	μs	33.95	**31.71** ⋆	32.29	33.86	**-6.6%**
Pro-Audio Tail Latency	Buffer P99 (↓)	μs	101.50	101.95	61.91	**58.42** ⋆	+0.4%
Audio Buffer Underruns	Total Xruns (↓)	count	3.67	**0.00** ⋆	0.00	0.00	**0 drops (Clean)**
Game Server Tick Latency	Physics P50 (↓)	μs	756.40	777.14	**690.73** ⋆	718.47	+2.7%
Game Server Frame Turnaround	Physics P99 (↓)	μs	1,563.91	1,865.02	**1,235.41** ⋆	1,387.65	+19.3%
Game Server Frame Drops	Desync Rate (↓)	%	0.019%	**0.000%** ⋆	0.000%	0.000%	**0.00% drops**
Real-Time POSIX Jitter	Avg Jitter (↓)	μs	24.17	**21.27** ⋆	23.90	28.43	**-12.0%**
Real-Time Jitter Max Spike	Worst Spike (↓)	μs	8,383.00	450.00	468.33	**280.67** ⋆	**-94.6%**
HFT Order Matching Rate	Throughput (↑)	ops/s	14,471,005	15,764,644	16,584,388	**18,800,975** ⋆	+8.9%
HFT Turnaround P50 Latency	Roundtrip P50 (↓)	μs	528.11	**222.79** ⋆	550.89	244.76	**-57.8%**
HFT Turnaround P99 Latency	Roundtrip P99 (↓)	μs	880.68	576.34	892.72	**441.02** ⋆	**-34.6%**
Production Redis Cache	GET Throughput (↑)	RPS	235,541.9	202,213.9	186,438.3	198,829.6	-14.1%
Production Redis Cache	GET Latency P99 (↓)	μs	249.67	319.00	319.00	399.00	+27.8%
Cloud API Gateway Fan-Out	Ingress Rate (↑)	req/s	1,275.61	887.95	1,066.03	1,001.99	-30.4%
Cloud API Gateway Latency	Roundtrip P50 (↓)	ms	33.83	49.58	39.34	41.75	+46.6%
Cloud API Gateway Tail	Roundtrip P99 (↓)	ms	76.97	100.78	98.36	108.00	+30.9%
Hackbench Process Context-Switch	Process Time (↓)	s	0.35	0.29	0.37	**0.21** ⋆	**-16.9%**
Hackbench Thread Context-Switch	Thread Time (↓)	s	0.60	0.67	0.85	0.85	+12.3%
Perf Pipe IPC Latency	Ping-Pong Latency (↓)	μs	16.06	**2.55** ⋆	6.19	4.36	**-84.1%**
Perf Pipe IPC Throughput	Ping-Pong Rate (↑)	ops/s	62,335	**392,691** ⋆	205,677	269,291	**+6.3** ×
Perf Messaging Storm	Wall Time (↓)	s	0.040	0.045	0.075	**0.036** ⋆	+11.7%
Schbench Wakeup P50 Latency	Wakeup P50 (↓)	μs	10.00	1,996.67	2,118.00	3,974.67	+199.7 ×
Schbench Wakeup P99 Tail	Wakeup P99 (↓)	μs	3,852.00	**2,007.33** ⋆	15,472.00	6,077.33	**-47.9%**
Schbench Request P50 Latency	Request P50 (↓)	μs	18,698.67	29,280.00	21,472.00	25,674.67	+56.6%
Schbench Request P99 Tail	Request P99 (↓)	μs	38,506.67	170,752.00	284,842.67	140,373.33	+343.4%
Schbench Service Rate	Throughput (↑)	req/s	1,634.37	712.97	714.00	766.57	-56.4%
Sysbench CPU Compute	Throughput (↑)	EPS	11,712.23	6,028.19	5,965.74	5,969.98	-48.5%
Sysbench Thread Fairness	Standard Deviation σ (↓)	σ	2,303.38	**46.48** ⋆	223.38	304.96	**-98.0%**
Sysbench Memory Bandwidth	Throughput (↑)	MiB/s	10,991.27	**12,394.80** ⋆	12,376.37	11,891.51	**+12.8%**
Sysbench Memory Operations	Operations Rate (↑)	ops/s	11,255,061	**12,692,281** ⋆	12,673,403	12,176,903	**+12.8%**
L3 Cache Miss Rate	Hardware PMU (↓)	%	6.40%	2.66%	1.55%	**1.36%** ⋆	**-58.5% Misses**
Hardware IPC Retirement	Instruction/Cyc (↑)	IPC	1.18	0.88	0.83	0.83	-25.2%
Network Loopback Throughput	iperf3 Rate (↑)	Gbps	57.58	**111.47** ⋆	38.23	62.03	**+93.6%**
Kernel Build (`make kernel/`)	Wall Time (↓)	s	3.29	6.03	5.67	5.83	+83.4%

Note: Values represent mean performance across 3 independent executions. (↓) indicates lower is better; (↑) indicates higher is better. Star (⋆) indicates best performance among evaluated schedulers. Kernel build wall time for OPTIMA (6.03s) reflects the barrier-free configuration, matching peer extensible schedulers (5.67–5.83s).

1) **Pro-Audio Real-Time DSP:** Simulates low-latency audio processing pipelines, tracking buffer turnaround latencies and buffer underruns (xruns).
2) **Game Physics Server:** Simulates a multi-threaded game server tick loop, measuring physics P50/P99 latency and frame desync rates.
3) **Real-Time POSIX Jitter (`cyclictest`):** Measures high-precision timer jitter and worst-case latency spikes under load.
4) **High-Frequency Trading (HFT):** Simulated limit order book matching engine measuring order matching throughput and roundtrip P50/P99 latency.
5) **Production Redis Cache:** Memory cache benchmark evaluating GET throughput and P99 latency under concurrent pipelining.
6) **Microservices Cloud API Gateway:** Fan-out HTTP ingress gateway measuring roundtrip P50/P99 request latency and ingress rate.
7) **Process and Thread Context Switching (`hackbench`):** Evaluates IPC scheduler overhead across process and thread groups.
8) **IPC Ping-Pong Latency (`perf bench sched pipe`):** Measures bidirectional pipe context-switch latency and operations rate.
9) **Messaging Storm (`perf bench sched messaging`):** Evaluates high-density IPC message exchange duration.
10) **Scheduler Wakeup and Request Latency (`schbench`):** Evaluates multi-threaded request-response latencies and wakeup P99 tails under worker contention.
11) **Compute Throughput & Thread Fairness (`sysbench cpu`):** Measures events per second (EPS) and per-thread standard deviation σ.
12) **Memory Bandwidth & Operations (`sysbench memory`):** Evaluates memory transfer rates (MiB/s) and operations per second.
13) **Hardware PMU Cache Miss Rates:** Hardware performance counters measuring L3 cache miss percentages.
14) **Hardware Instruction Retirement (IPC):** Measures instructions retired per CPU clock cycle.
15) **Network Loopback Throughput (`iperf3`):** High-bandwidth TCP socket loopback rate in Gbps.
16) **Linux Kernel Compilation (`make kernel/`):** Multi-threaded parallel compilation measuring wall-clock build time.

C. Performance Results

Table II reports the complete results across all 34 evaluated metrics for CFS/EEVDF, `scx_optima`, `scx_rusty`, and `scx_rlfifo`.

Algorithm 3 OPTIMA-ENQUEUE (Deadline Calculus & Preemption Kick)

Require: Task p, enqueue flags enq_flags
$tctx \leftarrow$ lookup_task_ctx(p)
if p is pinned **or** p is kernel thread **or** $tctx.dispatch_local$ **then**
  scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, $\tau_{\min}$, enq_flags); **return**
end if
{Domain Placement Target DSQ Selection (Heuristic Thresholds)}
if $tctx.weight \geq w_{\mathrm{RT}}$ **then**
  $target_dsq \leftarrow$ DSQ_PERF
else if $tctx.avg_runtime \geq 3\text{ms} \vee tctx.is_batch$ **then**
  $target_dsq \leftarrow$ DSQ_EFF
else
  $target_dsq \leftarrow$ is_perf(task_cpu) ? DSQ_SHARED_P : DSQ_SHARED_E
end if
$is_rt \leftarrow (tctx.weight \geq w_{\mathrm{RT}} \vee (\neg tctx.is_batch \wedge tctx.avg_runtime < 1\text{ms}))$
$now \leftarrow$ scx_bpf_now()
if is_rt **then**
  $\tau \leftarrow \tau_{\min}$; $deadline \leftarrow now$
else
  $\tau \leftarrow (tctx.avg_runtime \geq 3\text{ms})\ ?\ \tau_{\max} : \max(\frac{Q_{\text{base}}}{1+q}, \tau_{\min})$
  $v \leftarrow \mathrm{clamp}(p.\text{scx.dsq_vtime}, now, now + 2\tau)$
  $p_{\text{est}} \leftarrow \max(tctx.avg_runtime, \tau_{\min})$
  $\rho \leftarrow \frac{\max(tctx.weight, 10)}{p_{\text{est}}}$ {Execution density $\rho = w/p_{\text{est}}$}
  $\Delta v \leftarrow \mathrm{clamp}\left(\frac{\tau \cdot p_{\text{est}}}{10 \cdot \max(tctx.weight, 10)},\ \tau_{\min},\ \tau_{\max}\right)$ {Density advance $\tau \cdot \rho_{\text{ref}}/\rho$, $\rho_{\text{ref}} = 0.1\,\mu\text{s}^{-1}$}
  $deadline \leftarrow v + \Delta v$
end if
p.scx.dsq_vtime $\leftarrow deadline$
scx_bpf_dsq_insert_vtime($p, target_dsq, \tau, deadline, enq_flags$)
$idle_cpu \leftarrow$ scx_bpf_pick_idle_cpu(p.cpus_allowed, 0)
if $idle_cpu \geq 0$ **then**
  scx_bpf_kick_cpu($idle_cpu$, SCX_KICK_IDLE); **return**
end if
if is_rt **then**
  $oapf_cpu \leftarrow$ OPTIMA-OAPF($p, tctx, \mathcal{P}$)
  if $oapf_cpu \geq 0$ **then**
    scx_bpf_kick_cpu($oapf_cpu$, SCX_KICK_PREEMPT)
  end if
end if

Algorithm 4 OPTIMA-DISPATCH (Asymmetric Work-Stealing)

Require: Executing core cpu
if is_perf(cpu) **then**
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
else
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
end if

1) IPC Latency and Communication Throughput: In `perf bench sched pipe`, OPTIMA delivers an unprecedented **2.55 μs ping-pong latency**, representing an **84.1% latency reduction** compared to CFS (16.06 μs) and beating both `scx_rlfifo` (4.36 μs) and `scx_rusty` (6.19 μs). Throughput surges to **392,691 ops/s** (a **6.3× speedup** over CFS). This performance directly validates our synchronous IPC fast-path: by avoiding cross-CCX migration on `SCX_WAKE_SYNC`, cache lines remain hot in L1/L2 caches. In `iperf3` network loopback, OPTIMA achieves **111.47 Gbps**, nearly doubling CFS (57.58 Gbps) and outperforming `scx_rlfifo` (62.03 Gbps).

2) Real-Time Latency and Jitter Suppression: In POSIX real-time jitter evaluation (`cyclictest`), OPTIMA achieves the lowest average jitter (**21.27 μs**) and suppresses worst-case jitter spikes by **94.6%** (450 μs vs. 8,383 μs on CFS). In pro-audio DSP, OPTIMA achieves **31.71 μs** turnaround latency with **zero audio buffer underruns** (0.00 xruns vs. 3.67 on CFS). Similarly, in game server physics simulation, OPTIMA eliminates frame drops entirely (**0.000% desync rate**).

3) Software Compilation and Compute Throughput: By removing artificial work-stealing barriers and enabling idle E-cores to drain shared queues, OPTIMA achieves a kernel compile time of **6.03s**, closely matching peer extensible schedulers `scx_rusty` (5.67s) and `scx_rlfifo` (5.83s), though paying the expected BPF struct-ops trampoline tax compared to in-kernel CFS (3.29s). In `sysbench memory`, OPTIMA achieves highest-in-class bandwidth (**12,394.80 MiB/s**) and operation rates (**12,692,281 ops/s**), representing a **+12.8% gain** over CFS. Furthermore, thread fairness standard deviation σ drops by **98.0%** (46.48 vs. 2303.38 on CFS), demonstrating balanced cross-core workload distribution.

D. Ablation Study: Synchronous IPC Fast-Path

To isolate the microarchitectural contribution of the synchronous IPC zero-lag fast-path (`SCX_WAKE_SYNC` intercepted in Algorithm 1), we perform a controlled ablation study on the bidirectional pipe ping-pong and network loopback workloads. We compare full OPTIMA against an ablated variant (**OPTIMA-NoSyncFastPath**) in which synchronous wakeups fall back to standard core search across the scheduling domain.

As shown in Table III, disabling the synchronous IPC fast-path causes ping-pong latency to jump from 2.55 μs to 17.63 μs (matching CFS's 16.06 μs), while throughput collapses from 392,691 to 56,712 ops/s (-85.5%). Hardware performance counter traces reveal the root cause: without immediate local enqueue, 48.7% of wakeups migrate across cache complex boundaries, triggering inter-processor interrupts (IPIs) and cache-line invalidation. With the fast-path enabled, communicating thread pairs execute back-to-back on the same

TABLE III
ABLATION ANALYSIS OF THE SYNCHRONOUS IPC FAST-PATH

Evaluated Metric	Ablated (No-Sync)	Full OPTIMA	Delta / Gain
Pipe Ping-Pong Latency (↓)	17.63 μs	**2.55 μs**	**-85.5% (6.9×)**
Pipe Ping-Pong Throughput (↑)	56,712 ops/s	**392,691 ops/s**	**+592.4%**
Network Loopback Rate (↑)	58.21 Gbps	**111.47 Gbps**	**+91.5%**
Cross-CCX Migrations (↓)	48.7%	**0.2%**	**-99.6%**
L1 Data Cache Miss Rate (↓)	8.42%	**1.15%**	**-86.3%**

physical core or SMT sibling, retaining hot cache lines and driving L1 data misses down from 8.42% to 1.15%.

VI. DISCUSSION AND LIMITATIONS

Our comprehensive evaluation across 34 metrics reveals both significant microarchitectural strengths and principled trade-offs inherent to specialized eBPF scheduling on asymmetric multiprocessors. In this section, we analyze the underlying systems mechanisms governing these trade-offs and delineate the operational limitations of our approach.

A. Workload Trade-offs and Architectural Specialization

The empirical results demonstrate a clear bimodal performance profile:

- **Low-Latency IPC and Real-Time Robustness:** In workloads characterized by synchronous communication (`perf bench sched pipe`, `iperf3` socket loopback) and strict timing bounds (pro-audio DSP, game server ticks, POSIX jitter), OPTIMA achieves decisive improvements over CFS: an 84.1% reduction in pipe latency, a 94.6% reduction in jitter spikes, and zero audio underruns or frame drops. Relative to peer extensible schedulers, OPTIMA matches their zero-drop real-time robustness (0.00 xruns, 0.000% desync rate). In audio processing tail turnaround, OPTIMA achieves 101.95 μs P99 (closely tracking native CFS at 101.50 μs), while peer schedulers `scx_rusty` (61.91 μs) and `scx_rlfifo` (58.42 μs) demonstrate tighter tail bounds on this micro-benchmark. Concurrently, peer schedulers demonstrate advantages in specific isolated metrics: `scx_rusty` achieves faster game physics turnaround (1,235.41 μs P99 vs. OPTIMA's 1,865.02 μs and CFS's 1,563.91 μs), and `scx_rlfifo` registers the lowest peak jitter spike (280.67 μs vs. OPTIMA's 450.00 μs, though both represent a massive reduction over CFS's 8,383.00 μs spike). These variations reflect the trade-offs of distinct queue designs under pure real-time streaming.
- **High-Concurrency and Memory-Bound Trade-offs:** Conversely, in throughput-oriented database and proxy workloads (Production Redis, Cloud API Gateway), CFS outperforms OPTIMA by 14–30%. These applications exhibit thousands of short, non-blocking requests with highly volatile burst runtimes that challenge our EWMA burst estimation. Under rapid connection multiplexing, static or slow-converging EWMA values misclassify short batch tasks as compute-bound, intermittently routing them to Efficiency cores or incurring queue rebalancing delays.

B. Peer Preemption and the Schbench Latency Trade-off

A notable finding in Table II is the behavior under `schbench`: OPTIMA's wakeup P50 latency is 1,996.67 μs compared to CFS's 10.00 μs. Importantly, this $\sim$2 ms P50 latency floor is shared across all evaluated `sched_ext` schedulers (`scx_rusty` at 2,118.00 μs and `scx_rlfifo` at 3,974.67 μs). This phenomenon is an intrinsic consequence of dispatch queue scheduling in `sched_ext`: newly woken threads await executing threads' time slices to elapse before being dispatched from DSQs, whereas native CFS preempts executing threads nearly instantaneously (10 μs) based on virtual runtime lag (v_{runtime}).

However, under heavy request concurrency, aggressive peer preemption under CFS causes cascading context switches, expanding CFS's wakeup P99 tail to 3,852.00 μs and causing `scx_rusty`'s tail to explode to 15,472.00 μs. In contrast, OPTIMA holds its wakeup P99 tail down to **2,007.33 μs**—a remarkably tight distribution differing from P50 by only 10.66 μs. This stability is governed directly by **Rule 2 of the Online Admissible Preemption Filter (OAPF)**:

$$w_{\text{curr}}(c) \geq w_{\text{wake}} \implies \text{Prune candidate } c$$

Because all `schbench` worker threads share identical nice levels ($w = 100$), Rule 2 intentionally suppresses peer preemption among equal-priority threads. By guaranteeing that once a thread begins its execution quantum ($\tau_{\min} = 2$ ms) it runs without interruption, OPTIMA eliminates thrashing and achieves rock-solid wakeup tail latency predictability.

Nonetheless, this anti-thrashing design imposes a substantial trade-off on end-to-end request-response metrics under worker pool contention. When worker threads cannot preempt peers, queued request-handling tasks must wait for running threads to yield or exhaust full quanta. Consequently, OPTIMA's request P50 latency increases to 29,280.00 μs (+56.6% vs. CFS's 18,698.67 μs), request P99 tail latency rises to 170,752.00 μs (+343.4% vs. CFS's 38,506.67 μs, though outperforming `scx_rusty`'s 284,842.67 μs), and aggregate service throughput declines to 712.97 req/s (-56.4% vs. CFS's 1,634.37 req/s, matching `scx_rusty` at 714.00 req/s and `scx_rlfifo` at 766.57 req/s). This illustrates the fundamental duality: protecting thread quanta against peer preemption minimizes wakeup jitter but incurs significant request-level queueing delays during saturated request-response concurrency.

C. Sysbench Compute Throughput and Thread Fairness

In `sysbench cpu`, all three `sched_ext` schedulers cluster near ∼6,000 EPS (OPTIMA: 6,028.19 EPS, `scx_rusty`: 5,965.74 EPS, `scx_rlfifo`: 5,969.98 EPS) within the 12-thread isolated partition, whereas CFS achieves 11,712.23 EPS. This occurs because the pure compute loop of sysbench creates continuous core contention where CFS allows threads to unrestrictedly saturate all core pipelines without queue structure overheads. However, the critical differentiator lies in thread fairness: under CFS, individual thread event completions exhibit extreme variance ($\sigma = 2,303.38$), with some threads starved on slower cores while others surge ahead. Under OPTIMA, per-thread standard deviation drops by **98.0%** down to $\sigma = 46.48$. This demonstrates that OPTIMA's density-guided domain placement ensures uniform compute progress across asymmetric cores at the expense of uncoordinated thread racing.

D. Extensible Scheduler Overheads in Dynamic Process Lifecycles

In the parallel Linux kernel compilation benchmark (`make kernel/`), the wall-clock build time is 6.03s for OPTIMA, matching `scx_rusty` (5.67s) and `scx_rlfifo` (5.83s), but lagging native CFS (3.29s). In earlier iterations of OPTIMA, build time was 11.63s due to an artificial queue barrier that prevented idle Efficiency cores from stealing tasks enqueued for Performance cores. Removing this barrier and enabling unconstrained asymmetric work-stealing reduced compilation time by **48.2%** (11.63s → 6.03s).

However, all `sched_ext` schedulers remain ∼83% slower than CFS on this workload. The root cause is microarchitectural trampoline overhead: kernel compilation executes tens of thousands of short-lived compiler processes (`gcc`, `as`, `ld`). For each process creation and termination, `sched_ext` invokes BPF struct-ops hooks (`ops.init_task`, `ops.exit_task`, `ops.select_cpu`, `ops.enqueue`). While each invocation costs only a few hundred nanoseconds, the cumulative overhead across hundreds of thousands of process lifecycles imposes a measurable penalty compared to in-kernel CFS C code that optimizes process creation paths natively.

E. Microarchitectural Asymmetry and Hardware IPC Retirement

In Table II, OPTIMA registers a hardware IPC retirement of 0.88 (a -25.2% delta vs. CFS at 1.18). Importantly, peer extensible schedulers also exhibit lower retirement rates (both `scx_rusty` and `scx_rlfifo` register 0.83 IPC), demonstrating that this lower instruction retirement is not an isolated side effect of OPTIMA's density steering, but rather reflects general microarchitectural execution characteristics and memory-bound stall patterns across the benchmark suite under `sched_ext`. In OPTIMA, steering batch compute to narrower Efficiency pipelines does contribute to lower per-cycle retirement for those specific threads, but OPTIMA actually achieves slightly higher retirement (0.88) than peer schedulers (0.83) due to improved cache locality (reducing L3 cache misses by 58.5% down to 2.66% vs CFS's 6.40%).

Notably, in Table II, `scx_rlfifo` registers an even lower L3 miss rate (1.36% vs. 2.66% on OPTIMA and 6.40% on CFS). This occurs because `scx_rlfifo`'s flat, single-queue FIFO design preserves strictly sequential task execution without domain rebalancing or cross-queue work-stealing passes, whereas OPTIMA incurs a minor cache footprint from hierarchical DSQ classification and work-stealing checks across the asymmetric topology.

F. Limitations and Future Work

Several open challenges remain for extensible asymmetric scheduling:

1) **Hardware Platform Diversity:** Our evaluation focuses on an asymmetric architecture with Performance and Efficiency cores sharing L3 cache. Future work must validate OPTIMA on disparate architectures, including ARM big.LITTLE mobile SoCs, Intel hybrid desktop processors with Hardware Feedback Interface (HFI), and multi-socket NUMA servers.
2) **Energy and Thermal Dynamics:** While our density-guided placement allocates long-running batch tasks to E-cores to maximize parallel energy efficiency, explicit integration with Energy-Aware Scheduling (EAS) models and dynamic voltage/frequency scaling (DVFS) governors remains an active research direction.
3) **Dynamic Burst Estimation in Complex Runtimes:** For JIT-compiled languages and asynchronous actor systems where execution bursts vary non-linearly, incorporating compiler-assisted hinting or statistical quantile tracking into BPF maps would refine task density estimates.

VII. Conclusion

In this paper, we presented **OPTIMA**, an extensible operating system scheduler for asymmetric multiprocessors implemented in Linux `sched_ext` eBPF. By replacing empirical fair-share heuristics with formal combinatorial optimization paradigms—Smith's Rule WSPT density ordering, Online Admissible Preemption Filtering (OAPF), density-guided domain placement, and synchronous IPC zero-lag dispatch—OPTIMA delivers principled scheduling foundations. Rigorous isolated empirical evaluation across 16 workloads demonstrates up to a 6.3× increase in IPC throughput, an 84.1% reduction in pipe latency, a 94.6% reduction in worst-case real-time latency spikes, and zero real-time audio drops. Our transparent analysis of workload trade-offs elucidates the conditions under which extensible scheduling excels, characterizes trampoline overheads under heavy process fork/exec storms and connection multiplexing, and establishes a principled empirical foundation for future asymmetric multiprocessor schedulers.

References

[1] I. Molnar, "[PATCH] modular scheduler core and completely fair scheduler (CFS)," Linux Kernel Mailing List (LKML), Apr. 2007.

[2] P. Zijlstra, "sched: Earliest eligible virtual deadline first (EEVDF)," Linux Kernel Mailing List (LKML), 2023, merged in Linux Kernel 6.6.

[3] I. Stoica and H. Abdel-Wahab, "Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation," Department of Computer Science, Old Dominion University, Norfolk, VA, USA, Tech. Rep. TR-95-22, 1995.

[4] T. Heo, D. Vernet, A. Righi, and C. Min, "sched_ext: Bpf extensible CPU scheduling in the Linux kernel," in *Proceedings of the Linux Plumbers Conference (LPC)*, 2024, linux 6.12 Mainline Implementation.

[5] D. Hodges, "Scheduling at scale: eBPF schedulers with sched_ext," in *SREcon24 Europe/Middle East/Africa (SREcon24 EMEA)*. Dublin, Ireland: USENIX Association, Oct. 2024. [Online]. Available: https://www.usenix.org/conference/srecon24emea/presentation/hodges

[6] C. Min, "scx_lavd: Latency-aware virtual deadline scheduling for high-framerate interactive computing," Linux sched-ext Ecosystem and Igalia Technical Reports, Tech. Rep., 2024.

[7] A. Righi, T. Heo, D. Vernet, and sched-ext contributors, "sched-ext: scx_rlfifo and scx_rustland_core reference implementation and documentation," Linux Kernel sched-ext Repository, https://github.com/sched-ext/scx, 2024, official sched-ext Round-Robin / FIFO user-space scheduler baseline based on scx_rustland_core.

[8] J. T. Humphries, N. Natu, A. Chaugule, O. Weisse, B. Rhoden, J. Don, L. Rizzo, O. Rombakh, P. Turner, and C. Kozyrakis, "ghOSt: Fast & flexible user-space delegation of Linux scheduling," in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, ser. SOSP '21. New York, NY, USA: ACM, 2021, pp. 588–604.

[9] A. K. Parekh and R. G. Gallager, "A generalized processor sharing approach to flow control in integrated services networks: The single-node case," *IEEE/ACM Transactions on Networking*, vol. 1, no. 3, pp. 344–357, 1993.

[10] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, S. E. Gehrke, and C. G. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proceedings of the 17th IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 1996, pp. 288–299.

[11] J. Lelli, C. Scordino, L. Abeni, and D. Faggioli, "Deadline scheduling in the Linux kernel," *Software: Practice and Experience*, vol. 46, no. 6, pp. 821–839, 2016.

[12] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proceedings of the 19th IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 1998, pp. 4–13.

[13] G. Lipari and S. K. Baruah, "Greedy reclamation of unused bandwidth in constant bandwidth servers," in *Proceedings of the 12th Euromicro Conference on Real-Time Systems (ECRTS)*. IEEE, 2000, pp. 193–200.

[14] C. Kolivas, "The brain Fuck scheduler: BFS introduction," Web publication and Linux Kernel Archives, 2009.

[15] ——, "Multiple queue skiplist scheduler (MuQSS) for Linux," Web publication and Linux Kernel Archives, 2016.

[16] R. Kumar, K. I. Farkas, N. P. Jouppi, P. Ranganathan, and D. M. Tullsen, "Single-ISA heterogeneous multi-core architectures: The potential for processor power reduction," in *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, 2003, pp. 81–92.

[17] R. Kumar, D. M. Tullsen, P. Ranganathan, N. P. Jouppi, and K. I. Farkas, "Single-ISA heterogeneous multi-core architectures for multithreaded workload performance," in *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, 2004, pp. 64–75.

[18] D. Koufaty, D. Reddy, and S. Hahn, "Bias scheduling in heterogeneous multi-core architectures," in *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, ser. EuroSys '10. Paris, France: ACM, 2010, pp. 125–138.

[19] S. Blagodurov, S. Zhuravlev, M. Dashti, and A. Fedorova, "A case for NUMA-aware contention management on multicore systems," in *Proceedings of the 2011 USENIX Annual Technical Conference (USENIX ATC)*, Portland, OR, USA, 2011, pp. 1–15.

[20] S. Zhuravlev, S. Blagodurov, and A. Fedorova, "Addressing shared resource contention in multicore processors via scheduling," in *Proceedings of the 15th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2010, pp. 129–142.

[21] J. C. Sáez, M. Prieto, A. Fedorova, and S. Blagodurov, "A comprehensive scheduler for asymmetric multicore systems," in *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, ser. EuroSys '10. Paris, France: ACM, 2010, pp. 139–152.

[22] S. Eyerman, K. Du Bois, and L. Eeckhout, "Speedup stacks: Identifying scaling bottlenecks in multi-threaded applications," in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2012, pp. 145–155.

[23] S. Eyerman and L. Eeckhout, "System-level performance metrics for multiprogram workloads," *IEEE Micro*, vol. 28, no. 3, pp. 42–53, 2008.

[24] M. Rasmussen, P. Bellasi, D. Eggemann, and Q. Perret, "Energy aware scheduling (EAS) in the Linux kernel," Linux Kernel Documentation, Documentation/scheduler/sched-energy.rst, 2018.

[25] Intel Corporation, "Intel 64 and IA-32 architectures software developer's manual, volume 3b: Hardware feedback interface (HFI)," Santa Clara, CA, USA, 2021.

[26] R. Neri, "[PATCH v8 0/9] thermal/drivers/intel: Add support for intel hardware feedback interface," Linux Kernel Mailing List (LKML), 2022.

[27] H. Guo, Q. Chen, M. Guo, and L. Xu, "SAWS: Selective asymmetry-aware work-stealing for asymmetric multi-core architectures," in *Proceedings of the IEEE 18th International Conference on High Performance Computing and Communications (HPCC)*. IEEE, 2016, pp. 116–123.

[28] Q. Chen, M. Guo, and H. Guan, "LAWS: Locality-aware work-stealing for multi-socket multi-core architectures," in *Proceedings of the 28th ACM International Conference on Supercomputing (ICS)*, ser. ICS '14. New York, NY, USA: ACM, 2014, pp. 3–12.

[29] W. E. Smith, "Various optimizers for single-stage production," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 59–66, 1956.

[30] J. K. Lenstra, A. H. G. Rinnooy Kan, and P. Brucker, "Complexity of machine scheduling problems," *Annals of Discrete Mathematics*, vol. 1, pp. 343–362, 1977.

[31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman and Company, 1979.

[32] J. Bruno, E. G. Coffman, and R. Sethi, "Sequencing problems with partial ordered tasks and alternative processors," *Journal of the ACM*, vol. 21, no. 4, pp. 667–676, 1974.

[33] T. Kawaguchi and S. Kyan, "Worst case bound of an LRF schedule for the mean weighted flow-time problem," *SIAM Journal on Computing*, vol. 15, no. 4, pp. 1119–1129, 1986.

[34] L. A. Hall, A. S. Schulz, D. B. Shmoys, and J. Wein, "Scheduling to minimize average completion time: Off-line and on-line approximation algorithms," *Mathematics of Operations Research*, vol. 22, no. 3, pp. 513–544, 1997.

[35] N. Megow and A. S. Schulz, "On-line scheduling to minimize average completion time revisited," *Operations Research Letters*, vol. 32, no. 5, pp. 485–490, 2004.

[36] C. A. Phillips, C. Stein, and J. Wein, "Minimizing average completion time in the presence of release dates," *Mathematical Programming*, vol. 82, no. 1, pp. 199–223, 1998.

[37] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*, ser. Synthesis Lectures on Communication Networks. Morgan & Claypool Publishers, 2010, vol. 3, no. 1.

[38] ——, "Energy optimal control for time-varying wireless networks," *IEEE Transactions on Information Theory*, vol. 52, no. 7, pp. 2915–2934, 2006.

[39] L. Georgiadis, M. J. Neely, and L. Tassiulas, "Resource allocation and cross-layer control in wireless networks," *Foundations and Trends in Networking*, vol. 1, no. 1, pp. 1–144, 2006.